

8. Economic Analysis of the Solution

8.1 - Costing Scope and Method

This chapter estimates the cost of building and operating the FinOps platform. Its economic boundary differs from the cloud expenditure analyzed in chapter 6: a charge represented in a FOCUS file is an analytical input, whereas Databricks, GCP services and human work are resources used to produce the analysis. A cost estimate must identify its period, currency, environment and unit of work before figures from different settings can be compared.

Build cost covers design, implementation, tests, documentation and initial deployment. Operating cost covers recurring processing, storage, SQL queries, application use, monitoring and support. Human effort can be valued as recorded person-days by activity and role multiplied by an authorized loaded daily rate. Platform use can be valued from metered quantities and dated prices, with credits, discounts, taxes and any fixed charge shown separately. A shared resource requires an explicit allocation rule; otherwise its cost remains shared.

No complete effort log, approved labor rate or full Databricks and GCP billing extract is currently indexed for this project. Sections 8.2 and 8.3 therefore define the inputs and boundaries of a reproducible estimate. They do not report a project budget or return on investment. A scenario may vary volume, frequency or staffing, but its assumptions must remain separate from observed consumption.

8.2 - Build-Cost Scenarios

The build estimate groups effort by framing and architecture, dataset preparation, pipeline and contract implementation, analytical modeling, application development, security, testing and documentation. For each activity, the relevant record is the time actually spent by role and the scope of the resulting deliverable. Design and learning time should be visible as such; treating all PFE time as industrial delivery effort would obscure the difference between an academic prototype and a staffed deployment.

Cloud resources used during construction belong in the same build boundary when they can be attributed to development or testing. Their valuation needs metered compute, storage and query consumption with the applicable dated price. Credits may lower cash paid during the PFE, but the underlying usage remains relevant to a later deployment scenario. Resources that also serve routine operation must be allocated by a documented rule to avoid counting the same charge twice.

Three scopes can be compared once their inputs exist: the completed local prototype, the Databricks/GCP pilot and a prospective enterprise deployment. The last is a projection and would need assumptions about permissions, support, integration and procurement that are not

established by the prototype. No numerical build total is presented because the effort log, authorized rates and complete platform charges have not yet been assembled.

The organizational options in section 9.2 require a separate comparison at equivalent request volume and quality. External delivery, a dedicated FinOps Data Engineer and a mixed arrangement each involve implementation, review, handoff and possible rework. The student's report of delays identifies a question for measurement; it does not supply the time or cost data needed to rank these options.

8.3 - Operating-Cost Scenarios

8.3.1 Cost boundary and unit of analysis

The relevant economic object is the cost of operating the FinOps platform, not the cloud charges that the platform analyzes in chapter 6. A monthly estimate should include recurring ingestion and close runs, SQL Warehouse use, App hosting where billed, RAW and managed Delta storage, network charges where applicable, monitoring and human support. A run-level view is also needed to compare compute alternatives. The reference unit should be stated explicitly—for example, one historical billing month processed, one newly arrived Daily file, or one calendar month of service—because these workloads have different frequency and data volume.

Let n_i be the number of runs of workload type i , C_i their measured cost per comparable run, and C_{shared} the recurring cost not assigned to a run. The monthly estimate is $\sum(n_i \times C_i) + C_{shared}$. The shared term includes SQL Warehouse and application use, storage, attributable network and monitoring charges, and human support under a stated allocation rule.

This expression defines a method; no numerical result follows from it without consumption data. Build effort from section 8.2 is added only when computing total cost over a defined horizon. Credits or trial allowances affect cash paid, while usage before credits remains relevant to a future operating model.

8.3.2 Measuring Databricks and GCP consumption

The project contains a monitoring SQL script that links Job runs to their task timeline and Databricks billing records. For Serverless or dedicated Job Compute, a Job ID and Run ID can identify DBU usage and list-price cost directly when those metadata are present. Billing-system records may arrive after a run; an empty immediate query result must not be interpreted as zero usage. Report both consumed DBUs and the tariff/currency used for a monetary conversion.

The Belgium Classic templates use an All-Purpose cluster. Its billing records identify the cluster but may not identify the Job run. The monitoring script therefore estimates each run's Databricks share by the time overlap between the run window and cluster billing intervals. This attribution is reliable only if other notebooks and Jobs did not share the cluster during that window; otherwise their use is mixed into the estimate. The cluster ID, run interval, competing activity and allocation rule must be recorded. Classic total cost also includes the relevant GCP Compute Engine driver and worker charges, disks and network from GCP Cloud Billing. Databricks list-price DBUs alone understate Classic cost. For Serverless, the DBU charge already includes virtual-machine cost, so adding an equivalent VM charge would double count it.

Source: Databricks, Billable usage system table reference, Monitor job costs & performance with system tables and Best practices for cost optimization.

The RAW retention choice (section 4.4) also has a measurable consequence. Keeping one shared source for DEV and PROD avoids premature removal and supports replay, but GCS storage persists until a coordinated retention rule is introduced. DEV and PROD managed Delta tables and materialized datamarts create additional copies. Their volume and storage class should be measured rather than estimated solely from source-file size.

8.3.3 Scenario comparison

Three operating scenarios can be evaluated with the same model: a supervised PFE demonstration with manual runs, a pilot with scheduled Daily and monthly processing, and wider use with more users or more frequent queries. For each scenario, specify input volume, period, run frequency, concurrency, SQL Warehouse uptime, retention, App usage and human support. Sensitivity analysis should vary these drivers one at a time, particularly Daily backlog, monthly refresh frequency and Classic cluster idle time.

A Serverless-versus-Classic comparison requires the same source months, equivalent transformations and verified output equality, followed by repeated runs with recorded compute settings. Measure startup, total elapsed time, DBUs and full provider cost per processed month or million charge lines. The earlier Serverless STANDARD setting with `max_concurrent_runs = 4` was a development configuration; the Belgium Classic Daily Job instead limits concurrency to one. Comparing their unadjusted bills would conflate both compute mode and workload policy.

8.3.4 Current result and interpretation

The repository supplies the costing procedure and monitoring query, but the evidence register does not yet contain the Job and Run IDs, priced DBU outputs or GCP Cloud Billing extract needed for a numeric comparison. The current result is a defined measurement method and cost boundary. A comparison will require observed consumption normalized to a common workload, raw query outputs and dated price sources. A claim about return on investment would also need a comparable baseline and measured business benefit. Section 8.4 uses this method to assess possible changes to Databricks workloads.

8.4 - Applying FinOps to the FinOps Platform

8.4.1 Applying FinOps to the platform itself

The platform exists to improve the interpretation of cloud expenditure, but its own processing and reporting generate expenditure. This creates a second FinOps scope: the cost data being analyzed are the business object, while Databricks, GCP storage and compute are the cost of producing the analysis. Optimizing the latter should not weaken the former. A cheaper run that omits contract checks, delays monthly close beyond an agreed deadline or produces inconsistent datamarts would not be an acceptable improvement.

The first step is attribution. For each workload-historical backfill, one Daily file, monthly close, datamart refresh and dashboard query-record the source volume, Job or Warehouse identity, elapsed time, DBUs, applicable Databricks charge, GCP infrastructure charge and output quality. Useful unit measures are cost per processed billing month, cost per million charge lines and cost per successful Daily file, always paired with processing time and reconciliation status. These measures allow workload growth to be separated from a change in unit efficiency.

Databricks provides `system.billing.usage`, historical list prices and Job timeline tables for this analysis. Billing records may arrive after the run, so data freshness must be checked before treating a missing usage row as zero cost. The standard Job-cost queries do not attribute Jobs that run on an All-Purpose cluster as ordinary Job Compute; the project's Belgium Classic run therefore needs the cluster/time-overlap estimate described in section 8.3 and the associated GCP bill. The method and its attribution uncertainty must be stated alongside any comparison.

Source: Databricks, Billable usage system table reference and Monitor job costs & performance with system tables.

8.4.2 Workspace location and compute migration

According to the project owner's account, the platform was first exercised in a Frankfurt Databricks workspace with Serverless compute. After the team examined the platform's own consumption, the execution environment was rebuilt in a Belgium workspace using Classic compute. The current repository documents the latter configuration, but the earlier workspace and billing records still need to be archived for a fully traceable chronology. The available records do not establish the monetary saving attributable to location, compute mode or sizing individually. Earlier run identifiers, dates, SKU charges and GCP infrastructure charges must be retained before a numerical before-and-after claim is made.

The migration preserved the logical contract: the same external GCS RAW source and the catalog names `finops_raw`, `finops_dev`, `finops_prod` and `finops_ops`. The Classic deployment added a separate GCS bucket for Unity Catalog managed Delta storage and explicit catalog `MANAGED LOCATION` clauses, while the source bucket remained external and unchanged. A catalog's managed storage location is not the external FOCUS path read through `finops_raw.landing`. The storage change affected platform setup and compute-specific Job adapters, while common transformations and analytical products remained shared. The working completion note reports historical loading in the new workspace, subject to the evidence limits in section 7.4.

This migration is therefore a case of applying FinOps to the data platform itself, but it is not yet a controlled experiment. Region, Serverless versus Classic pricing, cluster size and lifecycle, Photon, concurrent-run policy and the timing of the runs may all affect the observed bill. Section 8.4.4 specifies how to separate these factors in a comparison.

8.4.3 Optimization levers relevant to this project

Lever	Project-specific question	Measurement and trade-off
Compute choice	Should repeated batch Jobs use Serverless, dedicated Classic Jobs Compute or the current Classic All-Purpose cluster?	Compare full cost and duration for identical output; include GCP VM/disk cost for Classic and idle time for All-Purpose.
Cluster lifecycle	How much paid time occurs before, between and after batch tasks?	Measure idle intervals and evaluate auto-termination or task-level Jobs Compute without missing the next processing window.
Sizing and Photon	Does a larger or Photon-enabled run reduce total cost per successful month, despite a different rate?	Repeat the same workload with one setting changed, record DBUs, GCP resources, elapsed time and output equality.
Job schedule and concurrency	Does processing one Daily file per run cause repeated startup or a slow backlog?	Compare controlled sequential catch-up with the current single-file Job; protect month-level writes and track freshness.
SQL Warehouse and datamarts	Are Warehouse uptime and full refresh of fourteen datamarts justified by dashboard use?	Measure query latency, refresh time, Warehouse active time and table freshness before changing auto-stop or refresh scope.
Data retention	How much do shared RAW objects and DEV/PROD Delta copies cost?	Measure stored bytes and access needs; retain sources until every consumer and audit requirement is satisfied.

These are hypotheses, not universal prescriptions. Databricks recommends matching compute to workload, using Jobs Compute for automated workloads where suitable, terminating idle Classic resources, and testing whether Photon improves cost per workload rather than duration alone. The project's current All-Purpose Classic configuration is a useful development and migration environment but merits measurement against alternatives once the pipeline is stable.

Source: Databricks, Compute selection recommendations and Best practices for cost optimization.

8.4.4 Controlled comparison and decision rule

An optimization claim requires an A/B experiment on the same billing files and code revision, with isolated destination tables and identical validation queries. Record region, runtime, cluster or Serverless mode, Photon, worker configuration, concurrency, Warehouse settings, cache state and run order. Repeat runs so that one unusually slow startup or temporary cloud condition does not determine the conclusion. The two Classic backfill templates labeled “with Photon” and “without Photon” do not alone prove a Photon effect if they also differ by environment or input; the setting must be isolated in a comparable test.

For each candidate, compare total provider cost, not only DBUs: Databricks consumption plus GCP Compute Engine, disks, network and storage where applicable, while avoiding double counting infrastructure already included in a Serverless SKU. Then compare cost with elapsed time, data freshness, job failure rate, and equality of Silver, Gold and datamart outputs. A candidate is preferable only if it meets the required quality and operational constraints and improves the chosen cost-performance measure. If the business requires faster availability, a more expensive but much faster configuration may still be justified; the service objective should be stated rather than silently assumed.

8.4.5 Current conclusion

The project already exposes several concrete opportunities: an All-Purpose Classic cluster may incur idle cost; the Daily Job processes one candidate per run; DEV and PROD retain separate Delta products; and the first cloud version refreshes all datamarts. At present, however, the evidence register lacks the priced run-level and GCP records needed to rank these levers or quantify a saving. The immediate FinOps action is to establish the measured baseline in section 8.3, then test one change at a time under the protocol in section 7.3. Chapter 9 will distinguish a measured recommendation from an untested deployment option.

References

Databricks. (n.d.-a). *Best practices for cost optimization*. <https://docs.databricks.com/gcp/en/lakehouse-architecture/cost-optimization/best-practices> (accessed 29 September 2026).

Databricks. (n.d.-b). *Billable usage system table reference*. <https://docs.databricks.com/gcp/en/admin/system-tables/billing> (accessed 29 September 2026).

Databricks. (n.d.-c). *Compute selection recommendations*. <https://docs.databricks.com/gcp/en/compute/choose-compute> (accessed 29 September 2026).

Databricks. (n.d.-d). *Monitor job costs & performance with system tables*. <https://docs.databricks.com/gcp/en/admin/system-tables/jobs-cost> (accessed 29 September 2026).